IUT Nantes
Pôle Sciences et technologie

*Département Informatique*

*BUT 1*

# Ressource R1.05 : Introduction aux bases de données et SQL

*Gilles Nachouki*

# Le plan I


Les limites à l'utilisation des fichiers

Définitions

Objectifs d'un SGBD

Fonctions d'un SGBD

- Définition et description des données
- Manipulation et restitution des données
- Contrôle

Principaux modèles de SGBDs

Le modèle de données Entité-Association

- Les concepts de base

Le modèle de données relationnel

Concepts de base

- Domaine
- Dépendance fonctionnelle

Normalisation

- Pourquoi normaliser ?
- Formes normles
  - 1 FN
  - 2 FN : DFs élémentaires à partir de la clé

# Le plan II

3 FN : DFs élémentaires et directes à partir de la clé

Passage Entité-Relation Relationnel

Algèbre relationnelle

Les opérateurs ensemblistes
- Union
- Différence
- Intersection
- Produit cartésien

Opérateurs relationnels
- Sélection
- Projection
- Jointure
- Jointure naturelle
- Semi-jointure
- Auto-jointure
- Jointure externe
- Division

Le langage SQL

# Le plan III

Le langage de définition
- la commande CREATE
- la commande ALTER
- la commande RENAME
- la commande DROP

Le langage de manipulation
- La commande INSERT
- la commande UPDATE
- la commande DELETE

Consultation de la base
- Définition
- Projection
- Sélection
- les opérateurs ensemblistes
- la jointure
- Les jointures normalisées
- Les requêtes imbriquées et corrélées
- Le regroupement

# Le plan IV

Tri des résultats

auto-incrémentation des clés

La table DUAL
- La table DUAL et les pseudo-colonnes
- Les séquences ORACLE

Références bibliographiques

# Les limites à l'utilisation des fichiers

1. L'utilisation de fichiers impose à l'utilisateur de connaître :
   - le mode d'accès (séquentiel, indexé, ...)
   - la structure physique des enregistrements
   - la localisation des fichiers.
2. Pour des applications nouvelles, l'utilisateur doit obligatoirement écrire de nouveaux programmes et il peut être amené à créer de nouveaux fichiers qui contiendront peut-être des informations déjà présentes dans d'autres fichiers.
3. Toute modification de la structure des enregistrements (ajout d'un champ par exemple) entraîne la réécriture de tous les programmes qui manipulent ces fichiers.

# Les limites à l'utilisation des fichiers

1. De telles applications sont :
   - rigides,
   - contraignantes
   - longues et coûteuse à mettre en oeuvre et à maintenir
2. Les données associées sont :
   - mal définies et mal désignées,
   - redondantes
   - peu accessibles de manière ponctuelle
   - peu fiables

# Définitions

## Base de données

Une base de données est un ensemble structuré de données (1) enregistrées sur des supports accessibles par l'ordinateur (2) pour satisfaire simultanément plusieurs utilisateurs (3) de manière sélective (4) en un temps opportun (5).

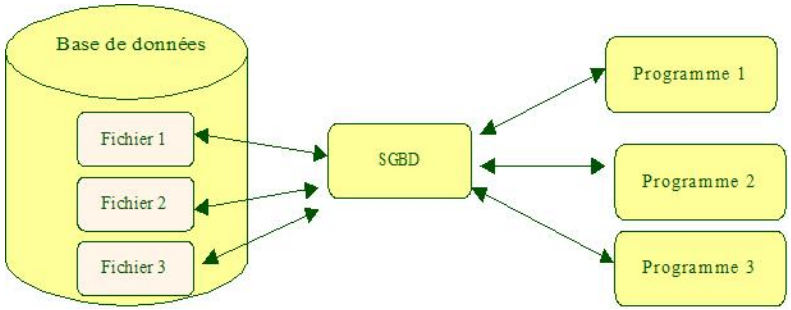
- (1) : Organisation et description de données
- (2) : Stockage sur disque
- (3) : Partage des données
- (4) : Confidentialité
- (5) : Performance

# Définitions

## Système de gestion de Base de données

Le logiciel qui permet d'interagir avec une BD est appelé Système de Gestion de Base de Données (SGBD). Un SGBD facilite :

- la gestion de données, avec une représentation intuitive simple sous forme de table par exemple
- la manipulation de données. On peut insérer, modifier les données et les structures sans modifier les programmes qui manipulent la base de données



# Indépendance physique/indépendance logique

## Indépendance physique :

La manipulation des données doit être facilitée en travaillant directement sur le schéma logique. On peut insérer, supprimer, modifier des données directement sur l'image logique. Le SGBD va s'occuper de faire le travail sur les fichiers physiques.

# Indépendance physique/indépendance logique

## Indépendance physique :

La manipulation des données doit être facilitée en travaillant directement sur le schéma logique. On peut insérer, supprimer, modifier des données directement sur l'image logique. Le SGBD va s'occuper de faire le travail sur les fichiers physiques.

## Indépendance logique :

Un même ensemble de données peut être vu différemment par des utilisateurs différents. Toutes ces visions personnelles des données doivent être intégrées dans une vision globale.

# Objectifs d'un SGBD

- Manipulation simple des données
- Efficacité des accès aux données
- Administration centralisée des données
- Cohérence des données
- Non redondance des données
- Partageabilité des données
- Sécurité des données
- Résistance aux pannes

# Définition et description des données

3 niveaux de description :

1. Conceptuel,
2. Externe,
3. Interne ou physique

# Définition et description des données

Niveau conceptuel :

Permet la description

- Des objets : exemple OUVRAGES, ETUDIANTS
- Des propriétés des objets (attributs) : exemple Titre de OUVRAGES
- Des liens entre les objets : exemple un OUVRAGE peut être emprunté par un ETUDIANT
- Des contraintes : exemple le nombre d'exemplaires d'un OUVRAGE est supérieur à zéro

# Définition et description des données

Niveau physique :

Description informatique des données et de leur organisation : en terme de fichiers, d'index, de méthodes d'accès, ...

# Définition et description des données

## Niveau physique :

Description informatique des données et de leur organisation : en terme de fichiers, d'index, de méthodes d'accès, ...

## Niveau externe :

Description des données vues par un utilisateur ( ou un groupe d'utilisateurs)

# Manipulation et restitution des données

Permet de :

1. Ajouter : saisir des données
2. Supprimer
3. Modifier
4. Interroger : rechercher des données via des requêtes

# Manipulation et restitution des données

Permet de :

1. Ajouter : saisir des données
2. Supprimer
3. Modifier
4. Interroger : rechercher des données via des requêtes

La manipulation des données est mise en oeuvre à l'aide d'un Langage de manipulation de Données (LMD). SQL (Structured Query Language) est le langage standard de manipulation de BD

# Contrôle

Un SGBD permet :

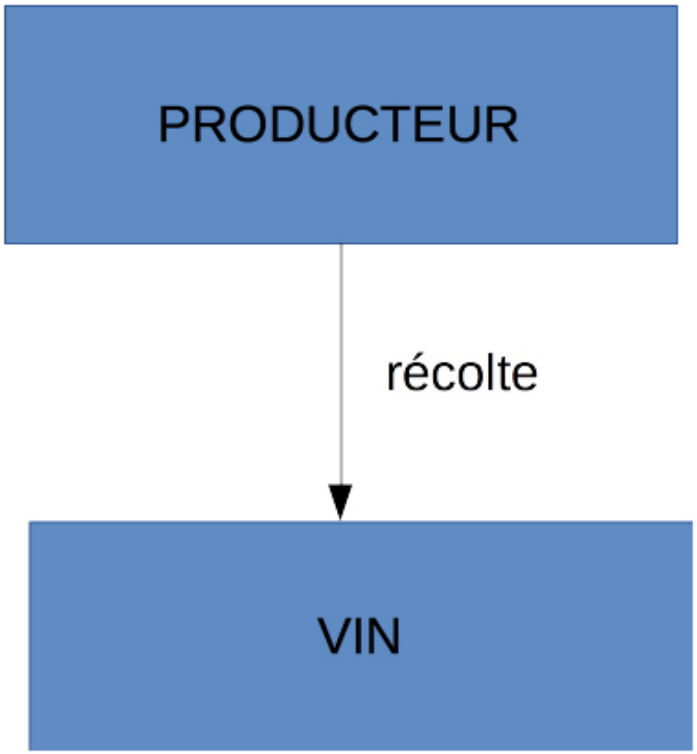
1. Le partage de données
2. L'intégrité des données
3. La confidentialité
4. La sécurité

# SGBD hiérarchique

Le schéma conceptuel peut être représenté par un graphe arborescent dont les noeuds correspondent aux types d'enregistrements et les arcs aux liaisons entre ces types d'enregistrements. Ce modèle utilise des pointeurs entre les différents enregistrements. Il s'agit du premier modèle de SGBD.

1. modèle père-fils
2. structure d'arbres
3. les noeuds correspondent aux types d'enregistrements et les arcs aux liaisons entre ces types d'enregistrements.
4. langage navigationnel
5. Impossible de présenter des liaisons de type n :m
6. Exemple : IMS

# SGBD hiérarchique



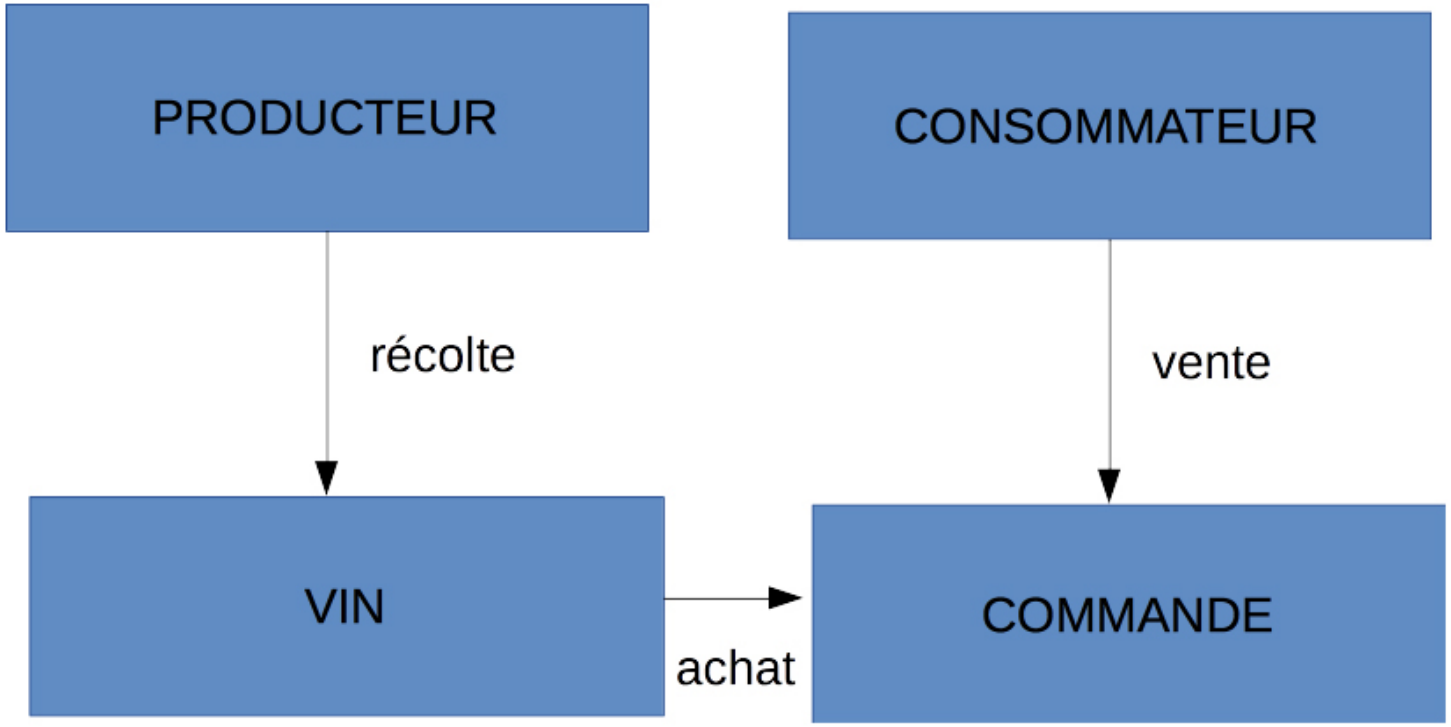
# SGBD réseau

## Definition

Le schéma conceptuel peut être représenté par un graphe dont le noeuds correspondent aux types d'enregistrements et les arcs aux liaisons entre ces types d'enregistrements. Un même type d'enregistrements peut être relié à deux pères de type différents.

1. modèle père-fils
2. structure de graphes
3. les noeuds correspondent aux types d'enregistrements et les arcs aux liaisons entre ces types d'enregistrements. l
4. langage navigationnel (rechercher le père, faire une opération sur tous les fils) au travers de pointeurs
5. Exemples : IDMS, IIDS, SOCRATE
6. Possible de présenter des liaisons de type n :m. Un même type d'enregistrements peut être relié à deux pères de type différents

# SGBD réseau


PRODUCTEUR
CONSOMMATEUR
récolte
vente
VIN
COMMANDE
achat

# SGBD relationnel

Les données et les relations qui les unissent sont organisées sous la forme de tableaux.La manipulation de ces données se fait selon la théorie mathématique des relations

# SGBD relationnel

Les données et les relations qui les unissent sont organisées sous la forme de tableaux.La manipulation de ces données se fait selon la théorie mathématique des relations

Exemple :

CONSOMMATEUR(no-cons, ...)
VIN(no-vin,...)
CONSOMME(, no-cons, no-vin)

# Historique

50 - 60 : Fichiers et méthodes d'accès (séquentiel, direct, séquentiel indexé).

62 - 63 : Apparition du concept de Base de Données.

65 - 70 : Conception des SGBD de 1ère Génération (modèles hiérarchique et réseau) :

- IMS d'IBM (hiérarchique),
- DS de General Electric (réseau) qui a servi de modèle de base aux propositions du groupe CODASYL.

70 - 85 : 2ème Génération des SGBD organisés sur le modèle relationnel : Plus de spécification des moyens d'accès aux données.

- MYSQL/PostgrSQL
- IBM DB2
- MS SQL
- ORACLE SQL, ...

et maintenant, ... Aujourd'hui Données plus hétérogènes (textes, sons, images, parole, ..),

- Bases de Données réparties,
- Bases de Données orientées objets,
- Bases de Connaissances et Systèmes Experts,
- Bases de données déductives,
- Bases XML, NoSQL, ...

# Introduction au modèle de données Entité-Association

Le Modèle Conceptuel des Données (MCD),permet la description statique du système d'information à l'aide des concepts d'entité et d'association.

# les objets ou entités ou types d' entités

- Une entité (ou type d' entités) est la représentation d'un objet pourvu d'une existence propre et conforme aux choix de gestion de l'entreprise.

# les objets ou entités ou types d' entités

- Une entité (ou type d' entités) est la représentation d'un objet pourvu d'une existence propre et conforme aux choix de gestion de l'entreprise.
- Une occurrence d'une entité est une instance (un représentant) de l'entité dans le monde réel.

# les objets ou entités ou types d' entités

- Une entité (ou type d' entités) est la représentation d'un objet pourvu d'une existence propre et conforme aux choix de gestion de l'entreprise.
- Une occurrence d'une entité est une instance (un représentant) de l'entité dans le monde réel.
- Une entité faible est une entité qui a besoin d'une autre entité pour pouvoir être définie.

# les objets ou entités ou types d' entités

- Une entité (ou type d' entités) est la représentation d'un objet pourvu d'une existence propre et conforme aux choix de gestion de l'entreprise.
- Une occurrence d'une entité est une instance (un représentant) de l'entité dans le monde réel.
- Une entité faible est une entité qui a besoin d'une autre entité pour pouvoir être définie.
- Une propriété (ou attribut) est une donnée élémentaire qui décrit l'entité.

# les objets ou entités ou types d' entités

- Une entité (ou type d' entités) est la représentation d'un objet pourvu d'une existence propre et conforme aux choix de gestion de l'entreprise.
- Une occurrence d'une entité est une instance (un représentant) de l'entité dans le monde réel.
- Une entité faible est une entité qui a besoin d'une autre entité pour pouvoir être définie.
- Une propriété (ou attribut) est une donnée élémentaire qui décrit l'entité.
- L'identifiant d'une entité est une propriété (ou un ensemble de plusieurs propriétés) de l'entité telle que, à chaque valeur de l'identifiant,correspond une et une seule occurence de l'entité.

# Règles de construction des entités

1. Toute propriété est atomique ;

# Règles de construction des entités

1. Toute propriété est atomique ;
2. Une propriété ne doit pas être "instable" pas "calculable" ;

# Règles de construction des entités

1. Toute propriété est atomique ;
2. Une propriété ne doit pas être "instable" pas "calculable" ;
3. Toute propriété d'une instance a au plus une valeur ;

# Règles de construction des entités

1. Toute propriété est atomique ;
2. Une propriété ne doit pas être "instable" pas "calculable" ;
3. Toute propriété d'une instance a au plus une valeur ;
4. Toute entité possède un identifiant ;

# Règles de construction des entités

1. Toute propriété est atomique ;
2. Une propriété ne doit pas être "instable" pas "calculable" ;
3. Toute propriété d'une instance a au plus une valeur ;
4. Toute entité possède un identifiant ;
5. Toute propriété dépend de tout l'identifiant ;

# Règles de construction des entités

1. Toute propriété est atomique ;
2. Une propriété ne doit pas être "instable" pas "calculable" ;
3. Toute propriété d'une instance a au plus une valeur ;
4. Toute entité possède un identifiant ;
5. Toute propriété dépend de tout l'identifiant ;
6. Toute propriété dépend (directement) de l'identifiant ;

# les associations ou types d' associations

- Une association (ou type d' association) est une relation que les règles de gestion établissent entre deux entités (ou plus).

# les associations ou types d' associations

- Une association (ou type d' association) est une relation que les règles de gestion établissent entre deux entités (ou plus).
- Une occurrence d'une association est une instance de l'association dans le monde réel.

# les associations ou types d' associations

- Une association (ou type d' association) est une relation que les règles de gestion établissent entre deux entités (ou plus).
- Une occurrence d'une association est une instance de l'association dans le monde réel.
- Une association peut posséder des propriétés.

# les associations ou types d' associations

- Une association (ou type d' association) est une relation que les règles de gestion établissent entre deux entités (ou plus).
- Une occurrence d'une association est une instance de l'association dans le monde réel.
- Une association peut posséder des propriétés.

# les associations ou types d' associations

- Une association (ou type d' association) est une relation que les règles de gestion établissent entre deux entités (ou plus).
- Une occurrence d'une association est une instance de l'association dans le monde réel.
- Une association peut posséder des propriétés.

**Règles à vérifier sur les associations :** *L'identifiant d'une association est la concaténation des identifiants des entités qu'elle relie.*

# Cardinalité d'une entité dans une association

S'exprime sous la forme d'un couple : (cardinalité minimale : cardinalité maximale)

cardinalité minimale : c'est le nombre minimal de fois où une occurrence d'un objet participe aux occurrences de l'association ; elle vaut très souvent 0 ou 1.

# Cardinalité d'une entité dans une association

S'exprime sous la forme d'un couple : (cardinalité minimale : cardinalité maximale)

cardinalité minimale : c'est le nombre minimal de fois où une occurrence d'un objet participe aux occurrences de l'association ; elle vaut très souvent 0 ou 1.

cardinalité maximale : c'est le nombre maximal de fois où une occurrence d'un objet participe aux occurrences de l'association ; elle vaut 1, un entier fixé ou n ("n" pour indiquer l'impossibilité de fixer une borne maximale).

# Détermination des cardinalités entre deux entités A et B

cmA : est obtenu en répondant à la question :"quand je considère un élément de l'entité A, combien d'éléments de l'entité B lui associés au minimum ?".

# Détermination des cardinalités entre deux entités A et B

cmA : est obtenu en répondant à la question :"quand je considère un élément de l'entité A, combien d'éléments de l'entité B lui associés au minimum ?".

cMA : est obtenu en répondant à la question :"quand je considère un élément de l'entité A,combien d'éléments de l'entité B lui sont associés au maximum ?".

# Détermination des cardinalités entre deux entités A et B

cmA : est obtenu en répondant à la question :"quand je considère un élément de l'entité A, combien d'éléments de l'entité B lui associés au minimum ?".

cMA : est obtenu en répondant à la question :"quand je considère un élément de l'entité A,combien d'éléments de l'entité B lui sont associés au maximum ?".

cmB Symétriquement, est obtenu en répondant à la question :"quand je considère un élément de l'entité B,combien d'éléments de l'entité A lui sont associés,au minimum ?".

# Détermination des cardinalités entre deux entités A et B

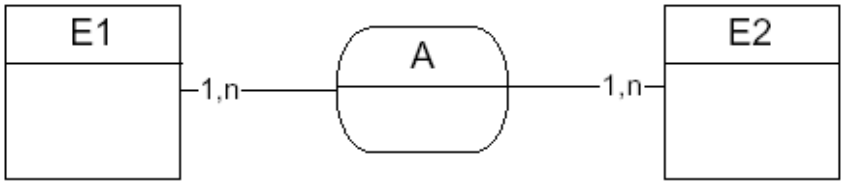
cmA : est obtenu en répondant à la question :"quand je considère un élément de l'entité A, combien d'éléments de l'entité B lui associés au minimum ?".

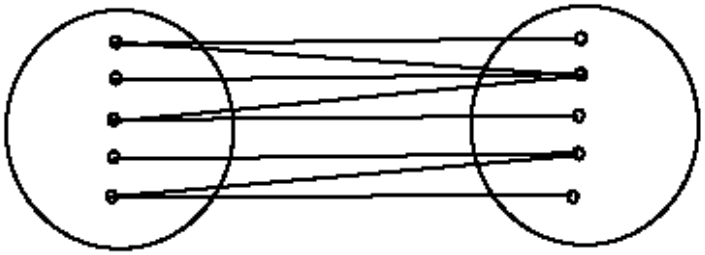
cMA : est obtenu en répondant à la question :"quand je considère un élément de l'entité A,combien d'éléments de l'entité B lui sont associés au maximum ?".

cmB Symétriquement, est obtenu en répondant à la question :"quand je considère un élément de l'entité B,combien d'éléments de l'entité A lui sont associés,au minimum ?".

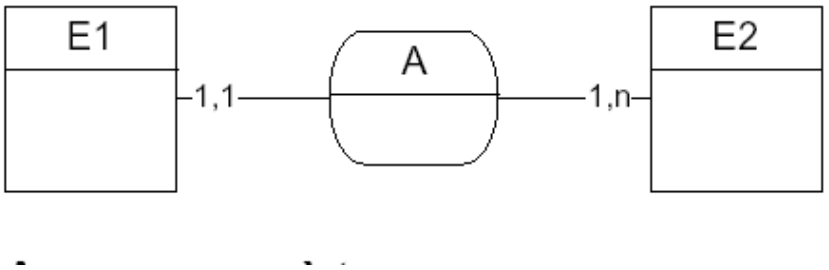
cMB est obtenu en répondant à la question :"quand je considère un élément de l'entité B,combien d'éléments de l'entité A lui sont associés au maximum ?".

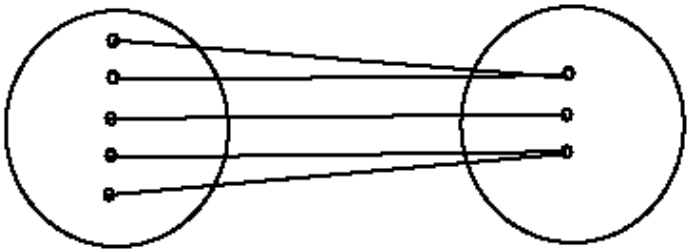
# Exemple : Cas tous à tous :



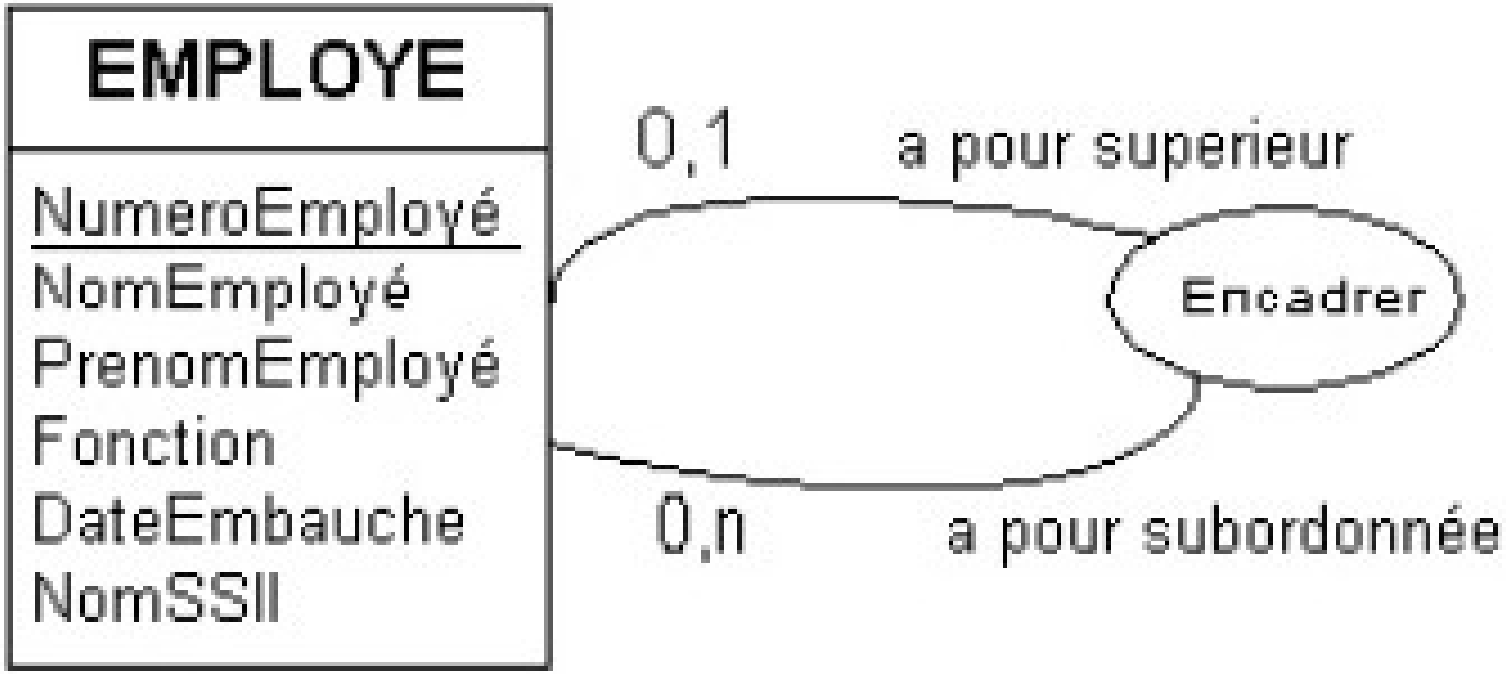
# Exemple : Cas un à tous :



# Association réflexive

Une association est qualifiée de réflexive quand elle matérialise une relation entre une entité et elle-même. On peut préciser les rôles sur les pattes de l' association.



# Vérification du MCD

Vérifier :

- les règles sur les entités ;

# Vérification du MCD

Vérifier :

- les règles sur les entités ;
- les règles sur les associations ;

# Vérification du MCD

Vérifier :

- les règles sur les entités ;
- les règles sur les associations ;
- les règles globales :

# Vérification du MCD

Vérifier :

- les règles sur les entités ;
- les règles sur les associations ;
- les règles globales :
  1. Une propriété ne figure qu'une fois dans le MCD ;

# Vérification du MCD

Vérifier :

- les règles sur les entités ;
- les règles sur les associations ;
- les règles globales :
  1. Une propriété ne figure qu'une fois dans le MCD ;
  2. Les propriétés calculées ne figurent pas dans le MCD (mais il faut s'assurer qu'avec le MCD on puisse les calculer) ;

# Vérification du MCD

Vérifier :

- les règles sur les entités ;
- les règles sur les associations ;
- les règles globales :
  1. Une propriété ne figure qu'une fois dans le MCD ;
  2. Les propriétés calculées ne figurent pas dans le MCD (mais il faut s'assurer qu'avec le MCD on puisse les calculer) ;
  3. On ne fait pas figurer les associations qui se déduisent par transitivité.

# Introduction au modèle de données relationnel

- Créé en 1970 par E.F Codd
- Fondé sur la théorie des ensembles

# Domaine :

Un domaine est un ensemble de valeurs (au sens théorie des ensembles)sur lequel une fonction ou une relation peut être définie.

# Domaine :

Un domaine est un ensemble de valeurs (au sens théorie des ensembles)sur lequel une fonction ou une relation peut être définie.

## Exemple

le domaine des noms, le domaine des âges,...
le domaine des entiers E={...-1,0,+1,+2,...}

# Attribut :

## Attribut :

Un attribut $A_i$ est une variable qui prend ses valeurs dans un ensemble $D_i$ appelé domaine.

## Exemple

noEtudiant :entier ; nomEtudiant :nom, âgeEtudiant :âge,...

# Relation

Soient alors $D_1, D_2, \cdots, D_n$ , une suite ordonnée de n domaines non forcément distincts, une relation R sur ces n domaines est une partie du produit cartésien $D_1 \times D_2 \times \cdots \times D_n$ .

# Relation

Soient alors $D_1, D_2, \cdots, D_n$ , une suite ordonnée de n domaines non forcément distincts, une relation R sur ces n domaines est une partie du produit cartésien $D_1 \times D_2 \times \cdots \times D_n$ .
Les éléments retenus dans le produit sont ceux qui correspondent à des informations dans le réel considéré.

# Relation

Soient alors $D_1, D_2, \cdots, D_n$ , une suite ordonnée de n domaines non forcément distincts, une relation R sur ces n domaines est une partie du produit cartésien $D_1 \times D_2 \times \cdots \times D_n$ .
Les éléments retenus dans le produit sont ceux qui correspondent à des informations dans le réel considéré.
Une relation est un ensemble de tuples $(d_1, d_2, \cdots, d_n)$ tels que :

$$d_1 \in D_1, d_2 \in D_2 \cdots, d_n \in D_n$$

## Exemple

| noEtudiant :entier | nomEtudiant :nom | âgeEtudiant :âge |
|---|---|---|
| 3 | Robert | 45 |

# Relation

Soient alors $D_1, D_2, \cdots, D_n$ , une suite ordonnée de n domaines non forcément distincts, une relation R sur ces n domaines est une partie du produit cartésien $D_1 \times D_2 \times \cdots \times D_n$ .
Les éléments retenus dans le produit sont ceux qui correspondent à des informations dans le réel considéré.
Une relation est un ensemble de tuples $(d_1, d_2, \cdots, d_n)$ tels que :

$$d_1 \in D_1, d_2 \in D_2 \cdots, d_n \in D_n$$

## Exemple

| noEtudiant :entier | nomEtudiant :nom | âgeEtudiant :âge |
|---|---|---|
| 3 | Robert | 45 |
| 5 | Raymonde | 37 |

# Relation I

## N-uplet d' une relation :

Assignation de valeurs à chacun des attributs de la relation. Chaque tuple d' une relation est désigné de façon unique par une valeur de clé.

### Exemple

| noEtudiant :entier | nomEtudiant :nom | âgeEtudiant :âge |
|---|---|---|
| 3 | Robert | 45 |

# Relation II

Exemple

| noEtudiant :entier | nomEtudiant :nom | âgeEtudiant :âge |
|---|---|---|
| 3 | Robert | 45 |
| 5 | Raymonde | 37 |

degré =3 (nombre d'attributs) ; cardinalité = 2 (nombre de tuples) ;

## Extension d' une relation :

L' extension d' une relation définit un état particulier de la relation à un instant donné.

Exemple

| noEtudiant :entier | nomEtudiant :nom | âgeEtudiant :âge |
|---|---|---|
| 3 | Robert | 45 |
| 5 | Raymonde | 37 |

# Relation III

A retenir

- Unicité des noms des attributs
- Indifférence de l'ordre des lignes et des colonnes

# Relation I

## Schéma de relation :

Le schéma d' une relation est une définition en intention d' une relation.Il contient :

- la liste de ses attributs et leur domaine,
- l' ensemble des contraintes d' intégrité associées à la relation

Ou, plus simplement
**doublet constitué de l' ensemble U des attributs et de l' ensemble F des contraintes d' intégrité**

$$\boxed{R = <U,F>}$$

# Relation II

## Exemple

ETUDIANT <U,F> U={noEtudiant :entier, nomEtudiant :nom, âgeEtudiant :âge}
F= ensemble de contraintes d'intégrité fonctionnelles

## Contrainte d' intégrité fonctionnelle

prédicat permettant de vérifier la validité des opérations de MAJ d' une relation.

# Relation I

## Relation normalisée :

Une relation est dite normalisée si ses attributs sont simples (une seule valeur pour chaque tuple)

## Exemple

Relation ETUDIANT :

| noEtudiant :entier | nomEtudiant | âgeEtudi | adresseEtudiant |
|---|---|---|---|
| 3 | Robert | 45 | 14,rue du Maines, 44000 Nantes |
| 5 | Raymonde | 37 | 12 rue des Roses 44000 Nantes |

Relation ETUDIANT :

| noEtudiant :entier | nomEtudiant | âgeEtudi | prénomEtudiant |
|---|---|---|---|
| 3 | Bidochon | 45 | Robert-Léon |
| 5 | Bidochon | 37 | Raymonde-Rose |

# Relation : Clé candidate / clé primaire / clé étrangère I

On admet généralement que les attributs et les tuples sont tous distincts. Il existe alors au moins un sous-ensemble minimal d' attributs de la relation dont les valeurs permettent de distinguer les tuples les uns des autres. Ce sous ensemble constitue une clé candidate de la relation.

- La valeur d'une clé candidate est donc distincte pour toutes les occurrences.
- Toute relation a au moins une clé candidate et peut en avoir plusieurs. Il ne peut donc jamais y avoir deux occurrences identiques au sein d'une relation.
- Les clés candidates d'une relation n'ont pas forcément le même nombre d'attributs.
- Lorsqu' il existe plusieurs clés candidates, l' une d' elles est choisie comme clé primaire.

Une clé étrangère d'une relation X est formée d'un ou plusieurs de ses attributs qui constituent une clé candidate dans une autre relation Y. La relation X est dite référençante et la relation Y est dite référencée.

# Exemple I

Etudiant (no-étudiant, nom, prénom,noInsee, adresse)
Matière(no-matière, libellé)
Note(no-étudiant, no-matière, note)

Exemples de clés

# Dépendance fonctionnelle I

## Définition :

Soit la relation $R(A, B, C)$, l' attribut $B$ est dit fonctionnellement dépendant de l' attribut $A$ si étant donné 2 n-uplets : <a1, b1, c1> et <a2,b2,c2>

$$a1 = a2 \Rightarrow b1 = b2$$

On la note

$$A \rightarrow B$$

A : source de la DF, B : cible de la DF

## Exemple :

no-Etudiant→ nomEtudiant
La source peut être un ensemble d' attributs :
(no-Etudiant,no-Matière)→ note

X → Y ou X détermine Y ou Y dépend de X

# Dépendance fonctionnelle

## Propriétés : Axiomes d'Armstrong

Soit une relation R (A,B,C,G)

- réflexivité : $A \rightarrow A$ ($\forall A$ )

# Dépendance fonctionnelle

## Propriétés : Axiomes d'Armstrong

Soit une relation R (A,B,C,G)

- réflexivité : $A \rightarrow A$ ($\forall A$ )
- augmentation : si $A \rightarrow B \Rightarrow A, C \rightarrow B \; \forall C$

# Dépendance fonctionnelle

## Propriétés : Axiomes d'Armstrong

Soit une relation R (A,B,C,G)

- réflexivité : $A \rightarrow A$ ($\forall A$ )
- augmentation : si $A \rightarrow B \Rightarrow A, C \rightarrow B \ \forall C$
- transitivité : si $A \rightarrow B$ et $B \rightarrow C \Rightarrow A \rightarrow C$

# Dépendance fonctionnelle

## Propriétés : Axiomes d'Armstrong

Soit une relation R (A,B,C,G)

- réflexivité : $A \rightarrow A$ ($\forall A$ )
- augmentation : si $A \rightarrow B \Rightarrow A, C \rightarrow B \ \forall C$
- transitivité : si $A \rightarrow B$ et $B \rightarrow C \Rightarrow A \rightarrow C$
- pseudo-transitivité : si $A \rightarrow B$ et $B, C \rightarrow D \Rightarrow A, C \rightarrow D$

# Dépendance fonctionnelle

## Propriétés : Axiomes d'Armstrong

Soit une relation R (A,B,C,G)

- réflexivité : $A \rightarrow A$ ($\forall A$ )
- augmentation : si $A \rightarrow B \Rightarrow A, C \rightarrow B \ \forall C$
- transitivité : si $A \rightarrow B$ et $B \rightarrow C \Rightarrow A \rightarrow C$
- pseudo-transitivité : si $A \rightarrow B$ et $B, C \rightarrow D \Rightarrow A, C \rightarrow D$
- union : si $A \rightarrow B$ et $A \rightarrow C \Rightarrow A \rightarrow B, C$

# Dépendance fonctionnelle

## Propriétés : Axiomes d'Armstrong

Soit une relation R (A,B,C,G)

- réflexivité : $A \rightarrow A$ ($\forall A$ )
- augmentation : si $A \rightarrow B \Rightarrow A, C \rightarrow B \ \forall C$
- transitivité : si $A \rightarrow B$ et $B \rightarrow C \Rightarrow A \rightarrow C$
- pseudo-transitivité : si $A \rightarrow B$ et $B, C \rightarrow D \Rightarrow A, C \rightarrow D$
- union : si $A \rightarrow B$ et $A \rightarrow C \Rightarrow A \rightarrow B, C$
- décomposition : si $A \rightarrow B, C \Rightarrow A \rightarrow B$ et $A \rightarrow C$

# Dépendance fonctionnelle

## Exemple : Axiomes d'Armstrong

Soit la relation R<U,F>
avec

$$U = \{A, B, C\}$$

$$F = \{A \rightarrow B, C; B \rightarrow C\}$$

Exercice : Déduire de F toutes les DFs pouvant être générées en appliquant les propriétés des DFs

# DF simple :

## DF simple :

La DF X $\rightarrow$ Y est dite simple si sa partie droite ne comporte qu'un seul attribut.

Exemple : dépendance simple

## DFs

1. no-étudiant $\rightarrow$ nom, prénom
2. idpromotion $\rightarrow$ libelléPromotion

## Simple/non-simple ?

1.
2.

# Dépendance élémentaire :

Définition :

La dépendance $X \rightarrow Y$ est dite élémentaire si $\nexists X' \subset X / X' \rightarrow Y$

# Dépendance élémentaire :

Définition :

La dépendance $X \rightarrow Y$ est dite élémentaire si $\nexists X' \subset X / X' \rightarrow Y$

Exemple : dépendance élémentaire

DFs

1. $no-étudiant, no-matière \rightarrow note$
2. $no-étudiant, no-matière \rightarrow nom - étudiant$

Elémentaire/non-élémentaire ?

1.
2.

# Dépendance directe :

## Définition

La dépendance $X \rightarrow Y$ est dite directe si $\nexists$ un attribut $Z$ de $R / X \rightarrow Z \wedge Z \rightarrow Y$

# Dépendance directe :

## Définition

La dépendance $X \rightarrow Y$ est dite directe si $\nexists$ un attribut $Z$ de $R / X \rightarrow Z \wedge Z \rightarrow Y$

Exemple : dépendance directe

## DFs

1. $no-étudiant, \rightarrow idpromotion$
2. $idpromotion \rightarrow$
   $libellé - promotion$
3. $no - étudiant \rightarrow$
   $libellé - promotion$

## Directe/non directe ?

1.
2.
3.

# Pourquoi normaliser : Anomalies de MAJ

## Exemple

solution 1 :

Relation ETUDIANT :

| noEtudiant | nomEtudiant | codePromo | libelléPromo |
|---|---|---|---|
| 3 | Robert | INFO | Informatique |
| 5 | Raymonde | INFO | Informatique |
| 7 | Kador | GEA | Gestion Entreprise |

# Pourquoi normaliser : Anomalies de MAJ

Exemple

solution 2 :

Relation ELEVE :

| noEtudiant | nomEtudiant | codePromo |
|---|---|---|
| 3 | Robert | INFO |
| 5 | Raymonde | INFO |
| 7 | Kador | GEA |

Relation PROMOTION :

| codePromo | libelléPromo |
|---|---|
| INFO | Informatique |
| GEA | Gestion Entreprise |

# Pourquoi normaliser : Anomalies de MAJ

## Exemple

solution 1 :

Relation ETUDIANT :

| noEtudiant | nomEtudiant | codePromo | libelléPromo |
|---|---|---|---|
| 3 | Robert | INFO | Informatique |
| 5 | Raymonde | INFO | Informatique |
| 7 | Kador | GEA | Gestion Entreprise |

ajout d'une nouvelle promo : GE

# Pourquoi normaliser : Anomalies de MAJ

## Exemple

solution 1 :

Relation ETUDIANT :

| noEtudiant | nomEtudiant | codePromo | libelléPromo |
|---|---|---|---|
| 3 | Robert | INFO | Informatique |
| 5 | Raymonde | INFO | Informatique |
| 7 | Kador | GEA | Gestion Entreprise |

modification du libellé de la promo Info : Cybersécurité

+Pourquoi normaliser : Anomalies de MAJ

# Exemple

solution 1 :

Relation ETUDIANT :

| noEtudiant | nomEtudiant | codePromo | libelléPromo |
|---|---|---|---|
| 3 | Robert | INFO | Informatique |
| 5 | Raymonde | INFO | Informatique |
| 7 | Kador | GEA | Gestion Entreprise |

suppression de l'étudiant Kador

# Pourquoi normaliser : Anomalies de MAJ

solution 2 :

Relation ELEVE :

| noEtudiant | nomEtudiant | codePromo |
|---|---|---|
| 3 | Robert | INFO |
| 5 | Raymonde | INFO |
| 7 | Kador | GEA |

Relation PROMOTION :

| codePromo | libelléPromo |
|---|---|
| INFO | Informatique |
| GEA | Gestion Entreprise |

insertion ajout d'une nouvelle promo : Méca

# Pourquoi normaliser : Anomalies de MAJ

solution 2 :

Relation ELEVE :

| noEtudiant | nomEtudiant | codePromo |
|---|---|---|
| 3 | Robert | INFO |
| 5 | Raymonde | INFO |
| 7 | Kador | GEA |

Relation PROMOTION :

| codePromo | libelléPromo |
|---|---|
| INFO | Informatique |
| GEA | Gestion Entreprise |

insertion ajout d'une nouvelle promo : Méca

modification modification du libellé de la promo Info : Cybersécurité

# Pourquoi normaliser : Anomalies de MAJ

solution 2 :

Relation ELEVE :

| noEtudiant | nomEtudiant | codePromo |
|---|---|---|
| 3 | Robert | INFO |
| 5 | Raymonde | INFO |
| 7 | Kador | GEA |

Relation PROMOTION :

| codePromo | libelléPromo |
|---|---|
| INFO | Informatique |
| GEA | Gestion Entreprise |

| | |
|---|---|
| insertion | ajout d'une nouvelle promo : Méca |
| modification | modification du libellé de la promo Info : Cybersécurité |
| suppression | suppression de l'étudiant Kador |

# 1FN : attributs simples

## 1FN

Une relation est dite en 1FN si ses attributs sont atomiques (simples et mono-valués). Un attribut est dit atomique s'il ne contient qu'une seule valeur pour un tuple donné, et donc s'il ne regroupe pas un ensemble de plusieurs valeurs.

## Exemple

Personne(nom, prénoms, date-naissance, adresse)

# 2 FN

| No-étudiant | Nom | No-matière | Note |
|---|---|---|---|
| 5 | Bidochon Robert | BD | 10 |
| 7 | Bidochon Raymonde | BD | 14 |
| 7 | Bidochon Raymonde | Info | 14 |
| 9 | Bidochon Kador | Info | 18 |

# 2 FN

| No-étudiant | Nom | No-matière | Note |
|---|---|---|---|
| 5 | Bidochon Robert | BD | 10 |
| 7 | Bidochon Raymonde | BD | 14 |
| 7 | Bidochon Raymonde | Info | 14 |
| 9 | Bidochon Kador | Info | 18 |

## 2FN

Une relation est dite en 2FN si tous les constituants non clé sont totalement dépendants des clés.
*Permet d' éliminer les attributs qui ne décrivent pas l' «objet» représenté par la relation*

# 3 FN

| No-étudiant | Nom | Promo | Libellé-promo |
|---|---|---|---|
| 5 | Robert | ID | Info décisionnelle |
| 7 | Raymonde | ID | Info décisionnelle |
| 9 | Kador | SILR | Info et Réseaux |

# 3 FN

| No-étudiant | Nom | Promo | Libellé-promo |
|---|---|---|---|
| 5 | Robert | ID | Info décisionnelle |
| 7 | Raymonde | ID | Info décisionnelle |
| 9 | Kador | SILR | Info et Réseaux |

## 3FN

Une relation est dite en 3FN si tous ses constituants non clés sont directement et totalement dépendants des clés.
*Permet d' éliminer des sous-relations incluses dans une relation.*

# Le Modèle logique de données : Passage du modèle entité-association au modèle relationnel

Les règles de passage :

Règle 1 : Une entité devient une relation. La clé de la relation est l'identifiant de l'entité

Règle 2 : Une association de dimension 2 avec cardinalités un à plusieurs se réécrit en : une clé étrangère dans la relation déduite de l'entité portant la cardinalité maxi 1.

Règle 3 : Une association de dimension 2 avec cardinalités plusieurs à plusieurs se réécrit en : (1) créant une relation particulière qui contient comme attributs les identifiants des 2 entités associées. Ces attributs constituent à eux 2 la clé primaire de la relation. Ils sont individuellement clés étrangères (2) ajoutant la ou les éventuelles propriétés de l'association à cette relation.

Règle 4 : Une association de dimension supérieure à 2 se réécrit selon la règle 3.

# Introduction à l'algèbre relationnelle I

## Langages de requêtes

Permettent de manipuler et de retrouver des données d'une base de données.

# Introduction

## L'algèbre relationnelle

Les opérateurs de base de l'algèbre relationnelle sont des opérateurs unaires ou binaires appliqués à des relations.

L'application de chaque opération produit en résultat une nouvelle relation.

# Exemple :

PRODUIT

| p | nom-p |
|---|---|
| p1 | cassis |
| p2 | champagne |
| p3 | huitre |
| p4 | moutarde |
| p5 | salade |
| p6 | cornichon |
| p7 | muscadet |

FOURNISSEUR

| f | nom |
|---|---|
| f1 | Bidochon |
| f2 | Lagaffe |
| f3 | Deschiens |

FOURNISSEURbis

| f | nom |
|---|---|
| f3 | Deschiens |
| f4 | Mortimer |
| f5 | Black |
| f6 | Achille Talon |

# Les opérateurs ensemblistes

## Définition

Les opérateurs ensemblistes correspondent aux opérateurs classiques de la théorie des ensembles, définis sur des tables de même forme, considérées comme des ensembles de tuples.

## Relations compatibles

Deux relations sont dites compatibles si :

- elles possèdent les mêmes noms d'attributs et ont les mêmes degrés,
- les attributs correspondants dans les deux relations sont définis sur les mêmes domaines. Les opérateurs d'union, de différence et d'intersection nécessitent des relations compatibles.

# Union

## Définition

L'union de deux relations compatibles $R1$ et $R2$ est une relation $R3$ formée par l'ensemble des tuples appartenant à $R1$, ou à $R2$ ou aux deux relations.

## Notation

$$\boxed{R3 = R1 \cup R2}$$

**Propriétés :**

**Commutatif** : $R1 \cup R2 = R2 \cup R1$

**Associatif** : $(R1 \cup R2) \cup R3 = R1 \cup (R2 \cup R3)$

# Exemple :

FOURNISSEUR

| f | nom |
|---|---|
| f1 | Bidochon |
| f2 | Lagaffe |
| f3 | Deschiens |

FOURNISSEURbis

| f | nom |
|---|---|
| f3 | Deschiens |
| f4 | Mortimer |
| f5 | Black |
| f6 | Achille Talon |

Fournisseur $\cup$ Fournisseurbis

| f | nom |
|---|---|
| f1 | Bidochon |
| f2 | Lagaffe |
| f3 | Deschiens |
| f4 | Mortimer |
| f5 | Black |
| f6 | Achille Talon |

# Différence

## Définition

La différence entre deux relations compatibles R1 et R2 (dans cet ordre) est une relation R3 formée par l'ensemble des tuples appartenant à R1, mais pas à R2.

## Notation

$$\boxed{\text{R3} = \text{R1 - R2}}$$

**Propriétés :**

$R1 - R2 \neq R2 - R1$

# Exemple :

FOURNISSEUR

| f | nom |
|---|---|
| f1 | Bidochon |
| f2 | Lagaffe |
| f3 | Deschiens |

FOURNISSEURbis

| f | nom |
|---|---|
| f3 | Deschiens |
| f4 | Mortimer |
| f5 | Black |
| f6 | Achille Talon |

Fournisseur - Fournisseurbis

| f | nom |
|---|---|
| f1 | Bidochon |
| f2 | Lagaffe |

# Intersection

## Définition

L'intersection de deux relations compatibles R1 et R2 est une relation R3 formée par l'ensemble des tuples appartenant à R1 et à R2

## Notation

$$\boxed{R3 = R1 \cap R2}$$

**Propriétés :**

**Commutatif** : $R1 \cap R2 = R2 \cap R1$

**Associatif** : $(R1 \cap R2) \cap R3 = R1 \cap (R2 \cap R3)$

# Exemple :

FOURNISSEUR

| f | nom |
|---|---|
| f1 | Bidochon |
| f2 | Lagaffe |
| f3 | Deschiens |

FOURNISSEURbis

| f | nom |
|---|---|
| f3 | Deschiens |
| f4 | Mortimer |
| f5 | Black |
| f6 | Achille Talon |

Fournisseur $\cap$ Fournisseurbis

| f | nom |
|---|---|
| f3 | Deschiens |

# Produit cartésien

## Définition

Le produit cartésien de deux relations $R1$ et $R2$ est une relation $T$ constituée de l'ensemble de tous les tuples $t$ tels que $t$ est le regroupement d'un tuple $r1$ de $R1$ et d'un tuple $r2$ de $R2$.

$$Card(T) = card(R1) * card(R2).$$

Degré (T) = degré (R1) + degré (R2)

## Notation

$$\boxed{\text{T} = \text{R1} \times \text{R2}}$$

**Propriétés :**

**Commutatif** : R1 $\times$ R2 = R2 $\times$ R1

**Associatif** : (R1 $\times$ R2) $\times$ R3 = R1 $\times$ (R2 $\times$ R3)

# Exemple :

PRODUIT

| p | nom-p |
|---|---|
| p1 | cassis |
| p2 | champagne |

FOURNISSEUR

| f | nom |
|---|---|
| f1 | Bidochon |
| f2 | Lagaffe |
| f3 | Deschiens |

Fournisseur $\times$ Fournisseurbis

| f | nom | p | nom-p |
|---|---|---|---|
| f1 | Bidochon | p1 | cassis |
| f1 | Bidochon | p2 | champagne |
| f2 | Lagaffe | p1 | cassis |
| f2 | Lagaffe | p2 | champagne |
| f3 | Deschiens | p1 | cassis |
| f3 | Deschiens | p2 | champagne |

# Sélection

## Définition

La sélection (ou restriction) est une opération unaire qui sélectionne un ensemble de lignes (tuples) d'une relation en fonction d'un critère de sélection (prédicat ou expression logique de prédicats).
Le résultat d'une sélection est une relation de même schéma que la relation initiale.

**Prédicat de sélection :**

- soit une formule simple = expression de la forme :A=a ou A=B, où A,B $\in$ schema(R) et a $\in$ dom(A)
- soit une expression de la forme F1 $\wedge$ F2, F1 $\vee$ F2, $\neg$ F ou $\neg$ (F1) où F1 et F2 sont des formules de sélection
- on peut étendre ces formules à d'autres opérateurs binaires : $<, >, \leqslant, \ldots$

## Notation

$$R3 = \sigma_{predicat}(R1)$$

# Sélection

## Exemple :

PRODUIT

| p | nom-p | couleur | origine |
|---|---|---|---|
| p1 | cassis | rouge | Dijon |
| p2 | champagne | blanc | Reims |
| p3 | huitre | vert | Brest |
| p4 | moutarde | jaune | Dijon |
| p5 | salade | vert | Nice |
| p6 | cornichon | vert | Dijon |
| p7 | muscadet | jaune | Nantes |

Liste des produits verts

PRODUIT

| p | nom-p | couleur | origine |
|---|---|---|---|
| p1 | cassis | rouge | Dijon |
| p2 | champagne | blanc | Reims |
| p3 | huitre | vert | Brest |
| p4 | moutarde | jaune | Dijon |
| p5 | salade | vert | Nice |
| p6 | cornichon | vert | Dijon |
| p7 | muscadet | jaune | Nantes |

# Sélection

Liste des produits verts

R $= \sigma_{couleur='vert'}$(PRODUIT)

R

| p | nom-p | couleur | origine |
|---|---|---|---|
| p3 | huitre | vert | Brest |
| p5 | salade | vert | Nice |
| p6 | cornichon | vert | Dijon |

# Sélection

Liste des produits de couleur vert et jaune

PRODUIT

| p | nom-p | couleur | origine |
|---|---|---|---|
| p1 | cassis | rouge | Dijon |
| p2 | champagne | blanc | Reims |
| p3 | huitre | vert | Brest |
| p4 | moutarde | jaune | Dijon |
| p5 | salade | vert | Nice |
| p6 | cornichon | vert | Dijon |
| p7 | muscadet | jaune | Nantes |

# Sélection

Liste des produits de couleur vert et d'origine 'Nice'

PRODUIT

| p | nom-p | couleur | origine |
|---|---|---|---|
| p1 | cassis | rouge | Dijon |
| p2 | champagne | blanc | Reims |
| p3 | huitre | vert | Brest |
| p4 | moutarde | jaune | Dijon |
| p5 | salade | vert | Nice |
| p6 | cornichon | vert | Dijon |
| p7 | muscadet | jaune | Nantes |

$$R = \sigma_{couleur='vert' \; and \; origine='Nice''}(\text{PRODUIT})$$

# Projection

## Définition

La projection est une opération unaire qui sélectionne un ensemble de colonnes d'une relation.
Le résultat d'une projection est une relation ayant au plus autant de lignes que la relation initiale.

## Notation

$$\boxed{\text{R3} = \pi_{A_1, A_2, \cdots A_k} \ (\ \text{R1})}$$

# Exemple :

Liste des nos de produits

PRODUIT

| p | nom-p | couleur | origine |
|---|---|---|---|
| p1 | cassis | rouge | Dijon |
| p2 | champagne | blanc | Reims |
| p3 | huitre | vert | Brest |
| p4 | moutarde | jaune | Dijon |
| p5 | salade | vert | Nice |
| p6 | cornichon | vert | Dijon |
| p7 | muscadet | jaune | Nantes |

$R1 = \pi_p$ ( PRODUIT)

# Exemple :

Couleur des produits

PRODUIT

| p | nom-p | couleur | origine |
|---|---|---|---|
| p1 | cassis | rouge | Dijon |
| p2 | champagne | blanc | Reims |
| p3 | huitre | vert | Brest |
| p4 | moutarde | jaune | Dijon |
| p5 | salade | vert | Nice |
| p6 | cornichon | vert | Dijon |
| p7 | muscadet | jaune | Nantes |

R2

| couleur |
|---|
| rouge |
| blanc |
| vert |
| jaune |

# Exemple :

Liste des no produits de couleur vert

PRODUIT

| p | nom-p | couleur | origine |
|---|---|---|---|
| p1 | cassis | rouge | Dijon |
| p2 | champagne | blanc | Reims |
| p3 | huitre | vert | Brest |
| p4 | moutarde | jaune | Dijon |
| p5 | salade | vert | Nice |
| p6 | cornichon | vert | Dijon |
| p7 | muscadet | jaune | Nantes |

Exemple :

R3 $= \pi_{p}\ (\ \sigma_{couleur='vert'}\ (\text{PRODUIT}))$

# Jointure

## Définition

La jointure est une opération binaire qui, appliquée à deux relations R1 et R2, produit une restriction du produit cartésien de ces deux relations. Lorsque le critère de sélection est l'égalité, on parle d'équijointure, sinon on parle de $\theta$-jointure.

**Propriétés :**

**Commutatif** : $R1 \underset{\text{prédicat}}{\bowtie} R2 = R2 \underset{\text{prédicat}}{\bowtie} R1$

**Associatif** : $(R1 \underset{\text{prédicat1}}{\bowtie} R2) \underset{\text{prédicat2}}{\bowtie} R3 = R1 \underset{\text{prédicat1}}{\bowtie} (R2 \underset{\text{prédicat2}}{\bowtie} R3)$

# Exemple

Liste des fournisseurs (no, nom) qui fournissent qq chose

FOURNISSEUR

| f | nom | remise | ville |
|---|---|---|---|
| f1 | Bidochon | 5 | Paris |
| f2 | Lagaffe | 7 | Paris |
| f3 | Deschiens | 3 | Reims |
| f4 | Mortimer | 6 | Dijon |
| f5 | Black | 10 | Brest |
| f6 | Achille Talon | 1 | Paris |

FOURNITURE

| f | p | qte |
|---|---|---|
| f1 | p1 | 1 |
| f1 | p4 | 1 |
| f1 | p6 | 2 |
| f2 | p2 | 1 |
| f3 | p2 | 6 |
| f4 | p6 | 3 |
| f4 | p5 | 7 |
| f1 | p5 | 8 |
| f4 | p4 | 2 |
| f3 | p4 | 1 |
| f2 | p4 | 1 |
| f5 | p3 | 10 |

$$R = \pi_{fournisseur.f,nom} \left( FOURNISSEUR \underset{f=f}{\bowtie} FOURNITURE \right.$$

R

| f | nom |
|---|---|
| f1 | Bidochon |
| f2 | Lagaffe |
| f3 | Deschiens |
| f4 | Mortimer |
| f5 | Black |

# Questions :

1Trouver le numéro des fournisseurs qui me fournissent au moins un article ;

FOURNISSEUR

| f | nom | remise | ville |
|---|---|---|---|
| f1 | Bidochon | 5 | Paris |
| f2 | Lagaffe | 7 | Paris |
| f3 | Deschiens | 3 | Reims |
| f4 | Mortimer | 6 | Dijon |
| f5 | Black | 10 | Brest |
| f6 | Achille Talon | 1 | Paris |

PRODUIT

| p | nom-p | couleur | origine |
|---|---|---|---|
| p1 | cassis | rouge | Dijon |
| p2 | champagne | blanc | Reims |
| p3 | huitre | vert | Brest |
| p4 | moutarde | jaune | Dijon |
| p5 | salade | vert | Nice |
| p6 | cornichon | vert | Dijon |
| p7 | muscadet | blanc | Nantes |

FOURNITURE

| f | p | qte |
|---|---|---|
| f1 | p1 | 1 |
| f1 | p4 | 1 |
| f1 | p6 | 2 |
| f2 | p2 | 1 |
| f3 | p2 | 6 |
| f4 | p6 | 3 |
| f4 | p5 | 7 |
| f1 | p5 | 8 |
| f4 | p4 | 2 |
| f3 | p4 | 1 |
| f2 | p4 | 1 |
| f5 | p3 | 10 |

$$R1 = \pi_f \ ( \text{FOURNITURE}))$$

# Questions :

2 Trouver le numéro des fournisseurs qui me fournissent p6 ;

FOURNISSEUR

| f | nom | remise | ville |
|---|---|---|---|
| f1 | Bidochon | 5 | Paris |
| f2 | Lagaffe | 7 | Paris |
| f3 | Deschiens | 3 | Reims |
| f4 | Mortimer | 6 | Dijon |
| f5 | Black | 10 | Brest |
| f6 | Achille Talon | 1 | Paris |

PRODUIT

| p | nom-p | couleur | origine |
|---|---|---|---|
| p1 | cassis | rouge | Dijon |
| p2 | champagne | blanc | Reims |
| p3 | huitre | vert | Brest |
| p4 | moutarde | jaune | Dijon |
| p5 | salade | vert | Nice |
| p6 | cornichon | vert | Dijon |
| p7 | muscadet | blanc | Nantes |

FOURNITURE

| f | p | qte |
|---|---|---|
| f1 | p1 | 1 |
| f1 | p4 | 1 |
| f1 | p6 | 2 |
| f2 | p2 | 1 |
| f3 | p2 | 6 |
| f4 | p6 | 3 |
| f4 | p5 | 7 |
| f1 | p5 | 8 |
| f4 | p4 | 2 |
| f3 | p4 | 1 |
| f2 | p4 | 1 |
| f5 | p3 | 10 |

$R2 = \pi_f \ ( \ \sigma_{p='p6'}(\text{FOURNITURE}))$

# Questions :

3 Trouver le numéro des fournisseurs qui ne me fournissent rien ;

FOURNISSEUR

| f | nom | remise | ville |
|---|---|---|---|
| f1 | Bidochon | 5 | Paris |
| f2 | Lagaffe | 7 | Paris |
| f3 | Deschiens | 3 | Reims |
| f4 | Mortimer | 6 | Dijon |
| f5 | Black | 10 | Brest |
| f6 | Achille Talon | 1 | Paris |

PRODUIT

| p | nom-p | couleur | origine |
|---|---|---|---|
| p1 | cassis | rouge | Dijon |
| p2 | champagne | blanc | Reims |
| p3 | huitre | vert | Brest |
| p4 | moutarde | jaune | Dijon |
| p5 | salade | vert | Nice |
| p6 | cornichon | vert | Dijon |
| p7 | muscadet | blanc | Nantes |

FOURNITURE

| f | p | qte |
|---|---|---|
| f1 | p1 | 1 |
| f1 | p4 | 1 |
| f1 | p6 | 2 |
| f2 | p2 | 1 |
| f3 | p2 | 6 |
| f4 | p6 | 3 |
| f4 | p5 | 7 |
| f1 | p5 | 8 |
| f4 | p4 | 2 |
| f3 | p4 | 1 |
| f2 | p4 | 1 |
| f5 | p3 | 10 |

$$R3 = \pi_f\ (\text{FOURNISSEUR}) - \pi_f\ (\text{FOURNITURE})$$

# Questions :

4 Trouver le numéro des fournisseurs qui me fournissent quelque chose d'autre que 'p6' ;

5 Trouver le numéro des fournisseurs qui me fournissent quelque chose, mais pas p6 ;

FOURNISSEUR

| f | nom | remise | ville |
|---|---|---|---|
| f1 | Bidochon | 5 | Paris |
| f2 | Lagaffe | 7 | Paris |
| f3 | Deschiens | 3 | Reims |
| f4 | Mortimer | 6 | Dijon |
| f5 | Black | 10 | Brest |
| f6 | Achille Talon | 1 | Paris |

PRODUIT

| p | nom-p | couleur | origine |
|---|---|---|---|
| p1 | cassis | rouge | Dijon |
| p2 | champagne | blanc | Reims |
| p3 | huitre | vert | Brest |
| p4 | moutarde | jaune | Dijon |
| p5 | salade | vert | Nice |
| p6 | cornichon | vert | Dijon |
| p7 | muscadet | blanc | Nantes |

FOURNITURE

| f | p | qte |
|---|---|---|
| f1 | p1 | 1 |
| f1 | p4 | 1 |
| f1 | p6 | 2 |
| f2 | p2 | 1 |
| f3 | p2 | 6 |
| f4 | p6 | 3 |
| f4 | p5 | 7 |
| f1 | p5 | 8 |
| f4 | p4 | 2 |
| f3 | p4 | 1 |
| f2 | p4 | 1 |
| f5 | p3 | 10 |

# Jointure naturelle

## Définition

C'est une équijointure qui se fait sur la clé d'une relation et la référence à cette clé dans l'autre relation. Les attributs de jointure (en double) ne sont représentés ici qu'une fois.

## Notation

$$\boxed{\text{R3} = R1 \bowtie R2}$$

# Exemple :

Liste des noms des fournisseurs et des produits qu'ils fournissent

FOURNISSEUR

| f | nom | remise | ville |
|---|---|---|---|
| f1 | Bidochon | 5 | Paris |
| f2 | Lagaffe | 7 | Paris |
| f3 | Deschiens | 3 | Reims |
| f4 | Mortimer | 6 | Dijon |
| f5 | Black | 10 | Brest |
| f6 | Achille Talon | 1 | Paris |

PRODUIT

| p | nom-p | couleur | origine |
|---|---|---|---|
| p1 | cassis | rouge | Dijon |
| p2 | champagne | blanc | Reims |
| p3 | huitre | vert | Brest |
| p4 | moutarde | jaune | Dijon |
| p5 | salade | vert | Nice |
| p6 | cornichon | vert | Dijon |
| p7 | muscadet | blanc | Nantes |

FOURNITURE

| f | p | qte |
|---|---|---|
| f1 | p1 | 1 |
| f1 | p4 | 1 |
| f1 | p6 | 2 |
| f2 | p2 | 1 |
| f3 | p2 | 6 |
| f4 | p6 | 3 |
| f4 | p5 | 7 |
| f1 | p5 | 8 |
| f4 | p4 | 2 |
| f3 | p4 | 1 |
| f2 | p4 | 1 |
| f5 | p3 | 10 |

# Exemple :

Liste des noms des fournisseurs et des produits qu'ils fournissent

| f | nom | p | qte |
|---|---|---|---|
| f1 | Bidochon | p1 | 1 |
| f1 | Bidochon | p4 | 1 |
| f1 | Bidochon | p6 | 2 |
| f2 | Lagaffe | p2 | 1 |
| f3 | Deschiens | p2 | 6 |
| f4 | Mortimer4 | p6 | 3 |
| f4 | Mortimer | p5 | 7 |
| f1 | Bidochon | p5 | 8 |
| f4 | Mortimer | p4 | 2 |
| f3 | Deschiens | p4 | 1 |
| f2 | Lagaffe | p4 | 1 |
| f6 | Achille Talon | p3 | 10 |

$R1 = FOURNISSEUR \bowtie FOURNITURE$

$R = \pi_{f,nom,p,qte}(R1)$

$R1 = FOURNISSEUR \underset{f=f}{\bowtie} FOURNITURE$ ou

# Semi-jointure

## Définition

Une semi-jointure est une jointure dont on ne garde que les attributs de l'une des deux relations jointes.
Une semi-jointure peut être assimilée à une restriction d'une relation à l'aide d'un attribut d'une autre relation.

## Notation

Semi-jointure droite

$$\boxed{\mathrm{R3} = R1 \underset{\text{prédicat}}{\ltimes} R2}$$

Semi-jointure gauche

$$\boxed{\mathrm{R3} = R1 \underset{\text{prédicat}}{\rtimes} R2}$$

# Exemple : :

Trouver les fournisseurs qui fournissent qqchose ;

FOURNISSEUR

| f | nom | remise | ville |
|---|---|---|---|
| f1 | Bidochon | 5 | Paris |
| f2 | Lagaffe | 7 | Paris |
| f3 | Deschiens | 3 | Reims |
| f4 | Mortimer | 6 | Dijon |
| f5 | Black | 10 | Brest |
| f6 | Achille Talon | 1 | Paris |

PRODUIT

| p | nom-p | couleur | origine |
|---|---|---|---|
| p1 | cassis | rouge | Dijon |
| p2 | champagne | blanc | Reims |
| p3 | huitre | vert | Brest |
| p4 | moutarde | jaune | Dijon |
| p5 | salade | vert | Nice |
| p6 | cornichon | vert | Dijon |
| p7 | muscadet | blanc | Nantes |

FOURNITURE

| f | p | qte |
|---|---|---|
| f1 | p1 | 1 |
| f1 | p4 | 1 |
| f1 | p6 | 2 |
| f2 | p2 | 1 |
| f3 | p2 | 6 |
| f4 | p6 | 3 |
| f4 | p5 | 7 |
| f1 | p5 | 8 |
| f4 | p4 | 2 |
| f3 | p4 | 1 |
| f2 | p4 | 1 |
| f5 | p3 | 10 |

# Exemple :

Trouver les fournisseurs qui fournissent qqchose ;

| f | nom | remise | ville |
|---|---|---|---|
| f1 | Bidochon | 5 | Paris |
| f2 | Lagaffe | 7 | Paris |
| f3 | Deschiens | 3 | Reims |
| f4 | Mortimer | 6 | Dijon |
| f5 | Black | 10 | Brest |

$$R = \text{FOURNISSEUR} \underset{f=f}{\ltimes} \text{FOURNITURE}$$

# Auto-jointure

## Définition

Une Auto-jointure est une jointure d'une relation avec elle-même.
La relation considérée doit donc préalablement être renommée :

Renommage de tables

$$R1 \leftarrow R$$

$$R2 \leftarrow R$$

# Exemple :

Trouver le no des fournisseurs qui pratiquent la même remise que f1 ;

FOURNISSEUR

| f | nom | remise | ville |
|---|---|---|---|
| f1 | Bidochon | 5 | Paris |
| f2 | Lagaffe | 7 | Paris |
| f3 | Deschiens | 3 | Reims |
| f4 | Mortimer | 6 | Dijon |
| f5 | Black | 10 | Brest |
| f6 | Achille Talon | 5 | Paris |

. A ←FOURNISSEUR ; B ← FOURNISSEUR ;

$$R = A_{f='f1'} \underset{\text{A.remise=B.remise}}{\rtimes} B$$

FOURNISSEUR

| f | nom | remise | ville |
|---|---|---|---|
| f1 | Bidochon | 5 | Paris |
| f2 | Lagaffe | 7 | Paris |
| f3 | Deschiens | 3 | Reims |
| f4 | Mortimer | 6 | Dijon |
| f5 | Black | 10 | Brest |
| f6 | Achille Talon | 5 | Paris |

# Jointure externe

## Définition

Une jointure externe est une jointure qui inclut dans la relation résultat les tuples de l'une ou l'autre des relations opérandes même s'ils ne vérifient pas la condition de jointure.
Ces tuples sont complétés par des valeurs nulles dans la relation résultat

Notation

jointure externe gauche

$$A \underset{\text{prédicat}}{⟕} B$$

jointure externe droite

$$A \underset{\text{prédicat}}{⟖} B$$

jointure externe généralisée

$$A \underset{\text{prédicat}}{⟗} B$$

# Exemple :

Lister les fournisseurs (no, nom, ...) et éventuellement les produits qu'ils fournissent ;

FOURNISSEUR

| f | nom | remise | ville |
|---|---|---|---|
| f1 | Bidochon | 5 | Paris |
| f2 | Lagaffe | 7 | Paris |
| f3 | Deschiens | 3 | Reims |
| f4 | Mortimer | 6 | Dijon |
| f5 | Black | 10 | Brest |
| f6 | Achille Talon | 1 | Paris |

PRODUIT

| p | nom-p | couleur | origine |
|---|---|---|---|
| p1 | cassis | rouge | Dijon |
| p2 | champagne | blanc | Reims |
| p3 | huitre | vert | Brest |
| p4 | moutarde | jaune | Dijon |
| p5 | salade | vert | Nice |
| p6 | cornichon | vert | Dijon |
| p7 | muscadet | blanc | Nantes |

FOURNITURE

| f | p | qte |
|---|---|---|
| f1 | p1 | 1 |
| f1 | p4 | 1 |
| f1 | p6 | 2 |
| f2 | p2 | 1 |
| f3 | p2 | 6 |
| f4 | p6 | 3 |
| f4 | p5 | 7 |
| f1 | p5 | 8 |
| f4 | p4 | 2 |
| f3 | p4 | 1 |
| f2 | p4 | 1 |
| f5 | p3 | 10 |

# Exemple :

Lister les fournisseurs (no, nom, ...) et éventuellement les produits qu'ils fournissent

| f | nom | p | qte |
|---|---|---|---|
| f1 | Bidochon | p1 | 1 |
| f1 | Bidochon | p4 | 1 |
| f1 | Bidochon | p6 | 2 |
| f2 | Lagaffe | p2 | 1 |
| f3 | Deschiens | p2 | 6 |
| f4 | Mortimer4 | p6 | 3 |
| f4 | Mortimer | p5 | 7 |
| f1 | Bidochon | p5 | 8 |
| f4 | Mortimer | p4 | 2 |
| f3 | Deschiens | p4 | 1 |
| f2 | Lagaffe | p4 | 1 |
| f5 | Black | p3 | 10 |
| f6 | Achille Talon | - | - |

$$R = FOURNISSEUR \mathbin{⟕}_{f=f} FOURNITURE$$

# Division

## Principe :

Partir d'une table dividende ( R(A,B)) que l'on divise à l'aide d'une table diviseur ( S(B)) pour obtenir un quotient ( T(A)) , c'est à dire une table résultat. Le résultat est calculé à partir des valeurs d'une colonne pour lesquelles la seconde colonne de la table dividende possèdent toutes les valeurs du diviseur.

## but :

La division relationnelle est capable de traiter les requêtes de style ¨les $\cdots$ tels que TOUS les $\cdots$

# Division

## Exemple :

Liste des fournisseurs qui fournissent tous les produits

FOURNITURE

| f | p |
|---|---|
| f1 | p1 |
| f1 | p4 |
| f1 | p6 |
| f2 | p2 |
| f3 | p2 |
| f4 | p6 |
| f4 | p5 |
| f1 | p5 |
| f4 | p4 |
| f3 | p4 |
| f1 | p3 |
| f5 | p3 |

PRODUIT

| p |
|---|
| p1 |
| P4 |
| p6 |

# Division

## Exemple :

Liste des fournisseurs qui fournissent tous les produits

FOURNITURE

| f | p |
|---|---|
| f1 | p1 |
| f1 | p4 |
| f1 | p6 |
| f2 | p2 |
| f3 | p2 |
| f4 | p6 |
| f4 | p5 |
| f1 | p5 |
| f4 | p4 |
| f3 | p4 |
| f1 | p3 |
| f5 | p3 |

PRODUIT

| p |
|---|
| p1 |
| P4 |
| p6 |

R = FOURNITURE ÷ PRODUIT

R

| f |
|---|
| f1 |

# Introduction au langage SQL

## Structured Query Language = Langage d'interrogation structuré

Permet de :

- créer,
- modifier
- supprimer

tables et enregistrements

## Langage normalisé mais avec des implémentations parfois différentes

- L4G (langage de quatrième génération)
- Inspiré du modèle d'algèbre relationnel de Codd

# Introduction

Trois versions normalisées de SQL :

- SQL1 (1989) :
- SQL2 (1992) : version étendue à trois niveaux de conformité (entry, intermediate, full), version de référence bien que quelques SGBD supportent une partie de SQL3
- SQL3 , norme étendue à la gestion d'objets complexes

*SQL est un standard de fait utilisé par tous les SGBDR : Oracle, DB2, Informix, MySQL, PostsgreSQL, Access, SQL Server, ...*

# Introduction

syntaxe SQL

- Séparateur d'instruction : point virgule
- Séparateur de mots-clés : espace, tab, fin de ligne
- Mots-clés du SQL non sensibles à la casse
- Noms d'objets (table, champ) sensibles à la casse
- Entourer de ' ' ou " " les noms d'objets comportant un espace
- Valeurs de chaînes et date entre ' ' ou " " dans les expressions
- Commentaires : commentaire sur plusieurs lignes /* commentaires */, commentaire sur une ligne : -

# Exemple

Le schéma relationnel d'une base de données est le suivant :

DRAGONS (*IdDragon*, Nom, Sexe, Longueur, NombreEcailles, CracheduFeu, ComportementAmoureux)

NOURRITURES (*IdProduit*, libelle, calories)

REPAS (*IdDragon, IdProduit*, Quantité)

*On suppose que :*

- ComportementAmoureux={timide, macho, sincère, superflu, autoritaire, ...}
- Force={un peu, beaucoup, passionnément, pas du tout}
- La longueur d'un dragon doit être inférieur à son nombre d'écailles
- Un dragon ne peut pas consommer plus de cinq produits ;

# la commande CREATE

Création simple

```
CREATE TABLE nom_table
        [(nom_col1 typeDonnées,
          nom_col2 typeDonnées,
          ...)]
  ;
```

# la commande CREATE : la base

Les types de données

1. Numériques : NUMBER, INTEGER, DECIMAL, FLOAT, NUMBER[(longueur,[précision])]
2. Caractères et chaines : CHAR(longueur) , VARCHAR(longueur), VARCHAR2(longueur), LONG(longueur texte)
3. date : DATE,
4. Grands objets binaires : RAW, BLOB, ...

# Principales fonctions pour manipuler des dates

- TO_CHAR(colonneDate [, format [, 'NLS_DATE_LANGUAGE=Langue']]) convertit une date en chaîne suivant un certain format dans un certain langage ;
- TO_DATE(chaîneCaractères [, format [, 'NLS_DATE_LANGUAGE=Langue']]) convertit une chaîne en date suivant un certain format dans un certain langage ;
- EXTRACT(YEAR | MONTH | DAY | HOUR | MINUTE | SECOND FROM expression-DATE | expressionINTERVAL) extrait une partie donnée d'une date ;

# la commande CREATE : les contraintes

CREATE TABLE [schéma.]nomTable
( colonne1 type1 [DEFAULT valeur1] [NOT NULL] [CONSTRAINT nomContrainte1 typeContrainte1]...,
[ colonne2 type2 [DEFAULT valeur2] [NOT NULL] ],
[CONSTRAINT nomContrainte2 typeContrainte2]...) ;

Les contraintes peuvent être déclarées de deux manières :

- en même temps que la colonne (contraintes monocolonnes),
- nommées, après la déclaration de tous les attributs

Quatre types de contraintes sont possibles :

# la commande CREATE : les contraintes

1. UNIQUE (colonne1 [,colonne2], ...)
2. PRIMARY KEY (colonne1 [,colonne2], ...)
3. FOREIGN KEY (colonne1 [,colonne2], ...) REFERENCES [schéma.]nomTablePere (colonne1 [,colonne2],...) [ON DELETE { CASCADE | SET NULL}]
4. CHECK (condition)

# la commande CREATE : les contraintes

Définition des clés :

1. PRIMARY KEY (colonne1, colonne2, ...)
2. FOREIGN KEY (colonne1, colonne2, ...)
   REFERENCES Nom_de_la_table_étrangère(colonne1,colonne2,...)

# la commande CREATE : les contraintes

Contraintes

- CONSTRAINT /* pour nommer la contrainte */
- NOT NULL
- DEFAULT /* valeur par défaut*/
- UNIQUE
- CHECK /* prédicat à vérifier */

# la commande CREATE

## Création avec insertion de données

```
CREATE TABLE nom_table
        [
            (nom_col1,nom_col2, ...)
        ]
AS SELECT ... ;
```

## Exemple

```
CREATE TABLE dragonsBis
as select * from Dragons ;
```

# la commande ALTER

ALTER

- ADD : Ajouter une colonne ou une contrainte
- MODIFY : Modifier une colonne
- DROP : Supprimer une colonne ou une contrainte
- DISABLE | UNABLE constraint

# la commande ALTER

```
ALTER TABLE table_name ADD column_name datatype;

ALTER TABLE table_name DROP COLUMN column_name;

ALTER TABLE table_name MODIFY COLUMN column_name datatype;

ALTER TABLE table_name MODIFY column_name datatype NOT NULL;

ALTER TABLE table_name
ADD CONSTRAINT MyUniqueConstraint UNIQUE(column1, column2...);

ALTER TABLE table_name
ADD CONSTRAINT MyUniqueConstraint CHECK (CONDITION);

ALTER TABLE table_name
ADD CONSTRAINT MyPrimaryKey PRIMARY KEY (column1, column2...);

ALTER TABLE table_name
DROP CONSTRAINT MyUniqueConstraint ;
```

# RENAME

RENAME ancien nouveau

# la commande DROP

DROP TABLE nom_table [ CASCADE | RESTRICT ];

# La commande INSERT

Insertion de n-uplets dans une table

```
INSERT INTO nom_table [(nom_col1,
                        nom_col2, ...)]
        VALUES (val1, val2...);
```

Exemple

```
insert into dragons values ('Smeagol','H',152,1857,'O','
insert into dragons values ('Birdurh','H',258,4787,'N','
```

# la commande UPDATE

Modification de n-uplets dans une table

```
UPDATE nom_table
        SET nom_col1 = {expression1 |
                        ( SELECT ...) }
WHERE predicat;
```

Exemple

```
update dragons set sexe = 'M' where sexe ='H';
```

# la commande DELETE

Suppression de n-uplets dans une table

```
DELETE FROM nom_table
        [WHERE predicat];
```

Exemple

```
Delete from Dragons
        where sexe = 'M';
```

# la commande SELECT

## Format général :

```
SELECT [DISTINCT| ALL]
   {*| <value exp.> [, <value exp.>]...}
   FROM relation [variable],
        relation [ variable]
     [WHERE <search condition>]
          [GROUP BY <attribute>
                   [,<attribute >]...]
          [HAVING <search condition>]
   [ORDER BY <attribute.>[.{ASC | DESC}]
         [,<attribute.>[.{ASC | DESC}] ]...]
```

# la commande SELECT

## EXPRESSION DE VALEURS

- Calculs arithmétiques
- Fonctions agrégats

## CONDITION DE RECHERCHE

- Sélection, projection, jointure
- Recherche textuelle
- Recherche par intervalle
- Recherche sur valeur nulle

# la commande SELECT

## Format général de la condition de recherche :

```
<search condition> ::= [NOT]
<nom-colonne> Theta constante
<nom-colonne> Theta <nom-colonne>
<nom-colonne> LIKE <modèle-de-chaîne>
<nom-colonne>  IN  <liste-de-valeurs>
<nom-colonne> Theta (ALL , ANY , SOME) <liste-de-valeurs>
EXISTS <liste-de-valeurs>
%UNIQUE <liste-de-valeurs>
<nom-colonne> BETWEEN constante AND constante
<search condition> AND , OR <search condition> avec
Theta ::= <, =, >, = , = , <>  ou !=
```

Remarque : <liste de valeurs> peut être dynamiquement déterminée par une requête

# La projection

```
SELECT [DISTINCT] nom_col1 [[as] alias1], nom_col2 [[as]
FROM  nom_table ;
```

# Exemple

DRAGONS (*IdDragon*, Nom, Sexe, Longueur, NombreEcailles, CracheduFeu, ComportementAmoureux)

NOURRITURES (*IdProduit*, libelle, calories)

REPAS (*IdDragon, IdProduit*, Quantité)

1. liste des dragons (dragon, sexe, crachedufeu)

# Exemple

DRAGONS (*IdDragon*, Nom, Sexe, Longueur, NombreEcailles, CracheduFeu, ComportementAmoureux)

NOURRITURES (*IdProduit*, libelle, calories)

REPAS (*IdDragon, IdProduit*, Quantité)

1. liste des dragons (dragon, sexe, crachedufeu)
2. liste des comportements amoureux

# La sélection

```
 SELECT *|liste-attributs
FROM  nom_table
WHERE predicat ;
```

# Sélection

## Les prédicats de sélection simples

Un prédicat simple est le résultat de la comparaison de deux expressions au moyen d'un opérateur de comparaison qui peut être :

$$=; \neq; \prec; \leq; \succ; \geq$$

ou encore :
IN : expr1 IN (expr2, expr3, ...)
LIKE : expr LIKE chaine (_ remplace exactement 1 caractère, % remplace un nombre quelconque de caractères)

## Les prédicats de sélection composés

AND, OR, NOT

# Exemple

DRAGONS (*IdDragon*, Nom, Sexe, Longueur, NombreEcailles, CracheduFeu, ComportementAmoureux)

NOURRITURES (*IdProduit*, libelle, calories)

REPAS (*IdDragon, IdProduit*, Quantité)

1. liste des dragons

# Exemple

DRAGONS (*IdDragon*, Nom, Sexe, Longueur, NombreEcailles, CracheduFeu, ComportementAmoureux)

NOURRITURES (*IdProduit*, libelle, calories)

REPAS (*IdDragon, IdProduit*, Quantité)

1. liste des dragons
2. liste des dragons masculins

# Exemple

DRAGONS (*IdDragon*, Nom, Sexe, Longueur, NombreEcailles, CracheduFeu, ComportementAmoureux)

NOURRITURES (*IdProduit*, libelle, calories)

REPAS (*IdDragon, IdProduit*, Quantité)

1. liste des dragons
2. liste des dragons masculins
3. liste des dragons masculins et qui crachent du feu

# Exemple

DRAGONS (*IdDragon*, Nom, Sexe, Longueur, NombreEcailles, CracheduFeu, ComportementAmoureux)

NOURRITURES (*IdProduit*, libelle, calories)

REPAS (*IdDragon, IdProduit*, Quantité)

1. liste des dragons
2. liste des dragons masculins
3. liste des dragons masculins et qui crachent du feu
4. liste des dragons dont la longueur est supérieure à 200

# Exemple

DRAGONS (*IdDragon*, Nom, Sexe, Longueur, NombreEcailles, CracheduFeu, ComportementAmoureux)

NOURRITURES (*IdProduit*, libelle, calories)

REPAS (*IdDragon, IdProduit*, Quantité)

1. liste des dragons
2. liste des dragons masculins
3. liste des dragons masculins et qui crachent du feu
4. liste des dragons dont la longueur est supérieure à 200
5. liste des dragons dont la longueur est comprise entre 100 et 200

# Exemple

DRAGONS (*IdDragon*, Nom, Sexe, Longueur, NombreEcailles, CracheduFeu, ComportementAmoureux)

NOURRITURES (*IdProduit*, libelle, calories)

REPAS (*IdDragon, IdProduit*, Quantité)

1. liste des dragons
2. liste des dragons masculins
3. liste des dragons masculins et qui crachent du feu
4. liste des dragons dont la longueur est supérieure à 200
5. liste des dragons dont la longueur est comprise entre 100 et 200
6. liste des dragons qui ne sont ni matchos ni timides

# Exemple

DRAGONS (*IdDragon*, Nom, Sexe, Longueur, NombreEcailles, CracheduFeu, ComportementAmoureux)

NOURRITURES (*IdProduit*, libelle, calories)

REPAS (*IdDragon, IdProduit*, Quantité)

1. liste des dragons
2. liste des dragons masculins
3. liste des dragons masculins et qui crachent du feu
4. liste des dragons dont la longueur est supérieure à 200
5. liste des dragons dont la longueur est comprise entre 100 et 200
6. liste des dragons qui ne sont ni matchos ni timides
7. liste des dragons dont le nom comprend la chaîne de caractères 'oc'

# Exemple

DRAGONS (*IdDragon*, Nom, Sexe, Longueur, NombreEcailles, CracheduFeu, ComportementAmoureux)

NOURRITURES (*IdProduit*, libelle, calories)

REPAS (*IdDragon, IdProduit*, Quantité)

1. liste des dragons
2. liste des dragons masculins
3. liste des dragons masculins et qui crachent du feu
4. liste des dragons dont la longueur est supérieure à 200
5. liste des dragons dont la longueur est comprise entre 100 et 200
6. liste des dragons qui ne sont ni matchos ni timides
7. liste des dragons dont le nom comprend la chaîne de caractères 'oc'
8. liste des dragons qui n'ont pas de taille définie (valeur nulle)

# Exemple

DRAGONS (*IdDragon*, Nom, Sexe, Longueur, NombreEcailles, CracheduFeu, ComportementAmoureux)

NOURRITURES (*IdProduit*, libelle, calories)

REPAS (*IdDragon, IdProduit*, Quantité)

1. liste des dragons
2. liste des dragons masculins
3. liste des dragons masculins et qui crachent du feu
4. liste des dragons dont la longueur est supérieure à 200
5. liste des dragons dont la longueur est comprise entre 100 et 200
6. liste des dragons qui ne sont ni matchos ni timides
7. liste des dragons dont le nom comprend la chaîne de caractères 'oc'
8. liste des dragons qui n'ont pas de taille définie (valeur nulle)
9. différence entre taille et nombre d'écailles de chaque dragon]

# les opérateurs ensemblistes

## INTERSECTION

SELECT ... INTERSECT SELECT ...

## SOUSTRACTION

SELECT ... MINUS SELECT ...

## UNION et UNION ALL

L'opérateur UNION permet d'éviter les duplicatas (comme DISTINCT dans un SELECT). UNION ALL ne les élimine pas.
SELECT ... UNION SELECT ...
SELECT ... UNION ALL SELECT ...

## PRODUIT CARTESIEN

SELECT ... FROM R1,R2

# la jointure

```
SELECT ...
FROM nom_table1, nom_table2...
WHERE predicat;
```

Liste des dragons (nom, sexe) et des produits qu'ils consomment

```
select dragons.dragon, sexe, produit from repas, dragons where dragons.DRA
```

# Auto-jointure

Ex : Liste des dragons plus grands que Birduth

```
select a.dragon from dragons a, dragons b where a.longueur > b.longueur and
```

# Jointure externe

Ex : dragons (nom, sexe) et éventuellement des produits qu'ils consomment

```
select dragons.dragon, sexe, produit from repas, dragons
where dragons.DRAGON=repas.DRAGON(+);
```

# Les jointures normalisées

Syntaxe

```
SELECT [ { DISTINCT | UNIQUE } | ALL ] listeColonnes
FROM nomTable1 [{ INNER | { LEFT | RIGHT | FULL } [OUTER
JOIN nomTable2 {ON condition | USING ( colonne1 [, colon
| { CROSS JOIN | NATURAL [{ INNER | { LEFT | RIGHT | FUL
JOIN nomTable2 } ...
WHERE condition ];
```

# Les jointures normalisées

Pourquoi ?

Lecture plus facile :
- Le type de jointure est spécifié directement dans la clause FROM ;
- La condition de jointure est dissociée des conditions de recherches : se situe maintenant dans la clause ON et non plus dans la clause WHERE

L'ancienne syntaxe est toujours valide

‘

# Les jointures normalisées

Jointure interne :

```
SELECT ... FROM <table gauche>
        [INNER]JOIN <table droite>
        ON <condition de jointure>
```

Ex :Liste des dragons (nom,sexe) et des produits qu'ils consomment

```
select dragons.dragon, sexe, produit from repas, dragons where dragons.DR
select dragons.dragon, sexe, produit from repas join dragons on dragons.D
```

# Les jointures normalisées

Jointure naturelle :

```
SELECT ... FROM <table gauche>
        NATURAL JOIN <table droite>
       [USING <noms de colonnes>]
```

Si les tables A et B ont deux noms de colonne en commun et que ces colonnes sont nommées x et y, l'instruction
SELECT * FROM A NATURAL JOIN B; équivaut à
SELECT * FROM A, B WHERE A.x = B.x and A.y=B.y;

# Les jointures normalisées Jointure naturelle : Exemple

Ex :dragons (nom, sexe) et produits qu'ils consomment

```
select dragons.dragon, sexe , produit from repas, dragon
select dragons.dragon, sexe, produit from repas join dra
select distinct dragon, sexe, produit from repas  natura
```

# Les jointures normalisées

Jointure externe :

```
SELECT ... FROM <table gauche> LEFT |
                               RIGHT |
                               FULL OUTER JOIN
        <table droite> ON condition de jointure
```

# Les jointures normalisées : Jointure externe : Exemple

Exemple : Liste des dragons (nom, sexe) et éventuellement des produits qu'ils c

1)

```
select dragons.dragon, sexe, produit from repas, dragons where dragons.DRA
```

2)

```
select dragons.dragon, sexe, produit from repas right outer join dragons or
```

# Les requêtes imbriquées et corrélées

## Les requêtes imbriquées

- Sous-requête ramenant une seule ligne
- Sous-requête ramenant plusieurs lignes : Les opérateurs permettant de comparer une valeur à un ensemble de valeurs sont :
  1. l'opérateur IN
  2. les opérateurs obtenus en ajoutant ANY ou ALL à la suite d'un opérateur de comparaison classique

$$=; \neq; \prec; \leq; \succ; \geq$$

  3. ANY : la comparaison est vraie si elle est vraie pour au moins un des éléments de l'ensemble.
  4. ALL : la comparaison sera vraie si elle est vraie pour tous les éléments de l'ensemble.
  5. EXISTS, NOT EXISTS

# Les requêtes imbriquées : Exemple

Liste des dragons plus grands que Birduth

```
select a.dragon from dragons a, dragons b
where a.longueur > b.longueur and b.dragon = 'Birduth';

select dragon from dragons
where  longueur > ANY (select longueur from dragons wher
```

# Les requêtes imbriquées : Exemple

dragons dont la longueur est = à la longueur d'un dragon de sexe féminin

# Les requêtes imbriquées et corrélées

## Sous-requêtes ramenant plusieurs colonnes

Il est possible de comparer le résultat d'une requête ramenant plusieurs colonnes à une liste de colonnes. La liste de colonnes figurera entre parenthèses à gauche de l'opérateur de comparaison.

# Les requêtes imbriquées et corrélées exemple

dragons dont la longueur et le nombre d'écailles sont = à ceux de Birduth

```
select a.dragon from dragons a, dragons b
where
a.longueur = b.longueur and
 a.nombreecailles = b.nombreecailles and
 b.dragon = 'Birduth'';

select dragon from dragons
where  (longueur,nombreecailles) in
(select longueur, nombreecailles
from dragons where dragon = 'Birduth');
```

# Les requêtes imbriquées et corrélées

## Sous-requêtes corrélées

SQL sait également traiter une sous-interrogation faisant référence à une colonne de la table de l'interrogation principale. Le traitement dans ce cas est plus complexe car il faut évaluer la sous-interrogation pour chaque ligne de l'interrogation principale.

# Exemple

Liste des dragons qui ne mangent pas

## Solution 1 : requête de base

```
Select dragon from dragons minus Select dragon from repas;
```

## Solution 2 : requête imbriquée

```
Select dragon from dragons where dragon not in (select dragon from repas);
```

## Solution 3 : requête imbriquée corrélée

```
Select dragon from dragons where not exists (select * from repas where repa
```

# la commande GROUP BY

DRAGONS

| dragon | sexe | longueur | nbEcailles | cracheFeu | comportementAmoureux |
|---|---|---|---|---|---|
| Smeagol | H | 152 | 1857 | O | macho |
| Birdurh | H | 258 | 4787 | N | timide |
| Negueth | F | 128 | 1582 | O | sincere |
| MissToc | F | 183 | 2781 | N | volage |
| Bolong | H | 213 | 2754 | O | macho |
| Miloch | H | 83 | 718 | O | timide |
| Nessie | H | 168 | 1721 | N | macho |
| Tarak | F | 123 | 851 | O | timide |

# Les principales fonctions d'agrégat

Les principales fonctions de calcul sur l'ensemble des valeurs d'une colonne sont les suivantes :

```
AVG([DISTINCT | ALL] expression)
COUNT(* | [DISTINCT | ALL] expression)
MAX([DISTINCT | ALL] expression)
MIN([DISTINCT | ALL] expression)
STDDEV([DISTINCT | ALL] expression)
SUM([DISTINCT | ALL] expression)
VARIANCE([DISTINCT | ALL] expression)
DISTINCT
```

Exemple : moyenne des longueurs des dragons

# la commande GROUP BY

```
SELECT    FROM
WHERE  ...
GROUP BY expr_1 ,  expr_2 ,    ;
```

Dans la liste des colonnes résultat d'un SELECT comportant une fonction de groupe, ne peuvent figurer que des caractéristiques de groupe, c'est-à-dire : soit des fonctions de groupe ; soit des attributs figurant dans le GROUP BY.

Ex : moyenne des longueurs des dragons par comportement amoureux

# Sélection des groupes

```
SELECT    FROM
WHERE  ...
GROUP BY expr_1, expr_2,
HAVING  ...
```

# Sélection des groupes : Exemples

moyenne des longueurs par comportement amoureux pour les dragons males

```
select COMPORTEMENTAMOUREUX, AVG (longueur)longueurMoyen
where sexe = 'H'
group by COMPORTEMENTAMOUREUX;
```

moyenne des longueurs par comportement amoureux pour les dragons males partagés par plus de deux drag

```
select COMPORTEMENTAMOUREUX, AVG (longueur)longueurMoyen
where sexe = 'H'
group by COMPORTEMENTAMOUREUX
 having count(dragon) >2;
```

# Tri des résultats

```
ORDER BY {nom_col1 | num_col1 [DESC] [, nom_col2 |
                     num_col2 [DESC] ,...]}
```

# Division

DRAGONS (*IdDragon*, Nom, Sexe, Longueur, NombreEcailles, CracheduFeu, ComportementAmoureux)

NOURRITURES (*IdProduit*, libelle, calories)

REPAS (*IdDragon, IdProduit*, Quantité)

Liste des produits qui sont consommés par tous les dragons

# Division solution naturelle

## en langage 'pseudo naturel

on recherche les produits /
$\forall$(dragon) $\exists$(repas(dragon, produit)
ou encore
on recherche les produits /
$\nexists$(dragon) $\nexists$(repas(dragon, produit)

## en SQL

```
select * from nourritures
    where not exists (select * from dragons
        where not exists (select * from repas
            where repas.produit = nourritures.produit an
                  repas.dragon = dragons.dragon));
```

# Exemples

DRAGONS (*IdDragon*, Nom, Sexe, Longueur, NombreEcailles, CracheduFeu, ComportementAmoureux)

AIME (*DragonAimant, DragonAimé*, Force),

NOURRITURES (*IdProduit*, libelle, calories)

REPAS (*IdDragon, IdProduit*, Quantité)

COMPOSE DE (*Composant,compose*, Quantité)

# Exemples

- liste des dragons qui mangent tous les produits
- quels sont les dragons qui mangent au moins deux produits ?
- sélectionner les deux dragons les plus longs

# La table DUAL

Oracle fournit une table appelée « DUAL » qui se compose d'une ligne et d'une colonne qui est utilisée pour tester des fonctions ou effectuer des calculs rapides.

DESC DUAL

| Table | Column | Type de données |
|---|---|---|

# La table DUAL

La table DUAL permet par exemple de tester les pseudo-colonnes fournies par ORACLE :

## date du jour

```
SELECT SYSDATE FROM DUAL;
/* utilisateur courant */
SELECT user FROM DUAL;
/* id physique */
SELECT rowid FROM DUAL;
/*no ligne */
SELECT rownum FROM DUAL;
```

# Les séquences ORACLE

Une séquence est un objet ORACLE qui permet de gérer une suite de nombres entiers. Elle permet en particulier de générer des clés uniques dans des tables de la base.

# Affectation automatique d'un numéro lors de l'insertion d'un nouveau fournisseur

```
/*Création d'une séquence*/
CREATE SEQUENCE nouveaufournisseur ;
/* création du trigger permettant d'affecter un numéro au nouveau
fournisseur créé */
CREATE OR REPLACE TRIGGER incrementfournisseur
BEFORE INSERT ON fournisseur
FOR EACH ROW
BEGIN
SELECT nouveaufournisseur.NEXTVAL INTO :NEW.f FROM DUAL ;
END ;
```

# Pour plus d'infos ...


- Ramakrishnan R. et al.,
  Database management system,
  2003,McGraw-Hill
- Soutou C., Teste O.,
  SQL pour ORACLE,
  2004,Eyrolles
- Boudjlida N.,
  Gestion et administration de bases de données Application àSYBASE et ORACLE,
  2003,DUNOD